

ELI — Diagrams

ELI v2.3.10

August 2026

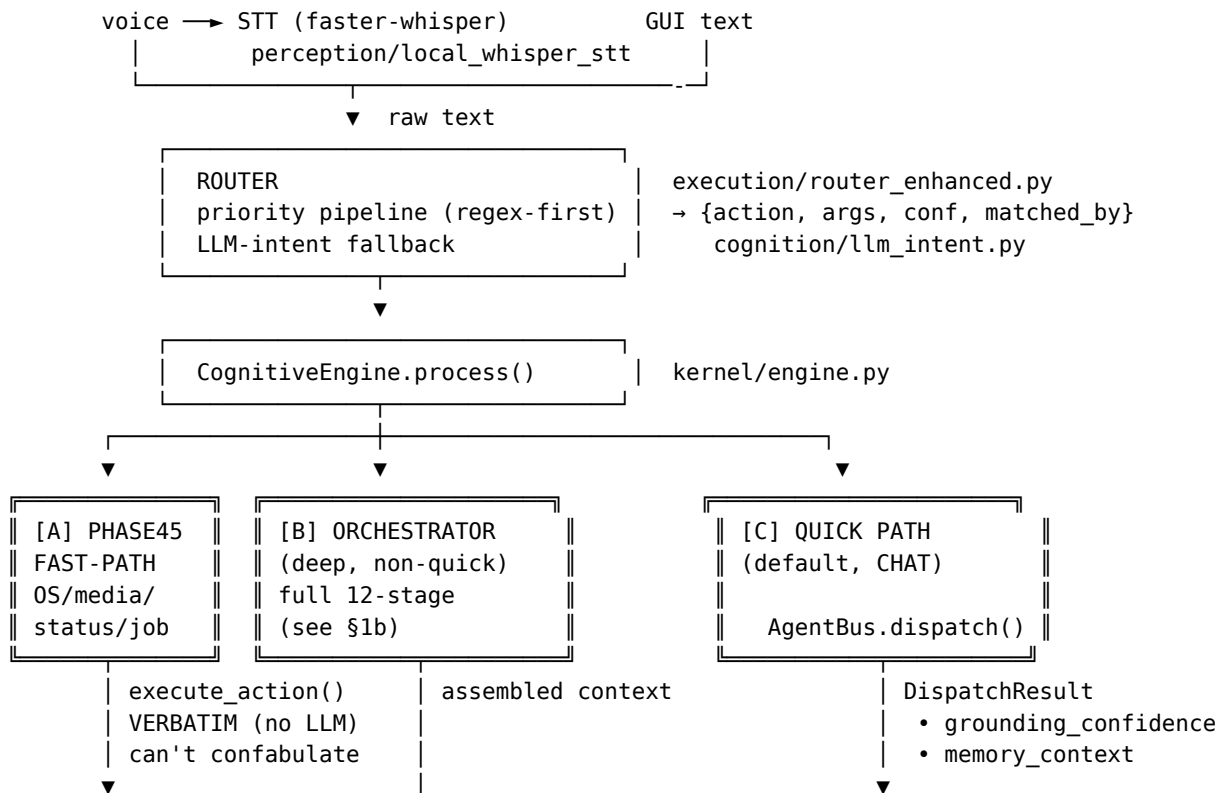
Contents

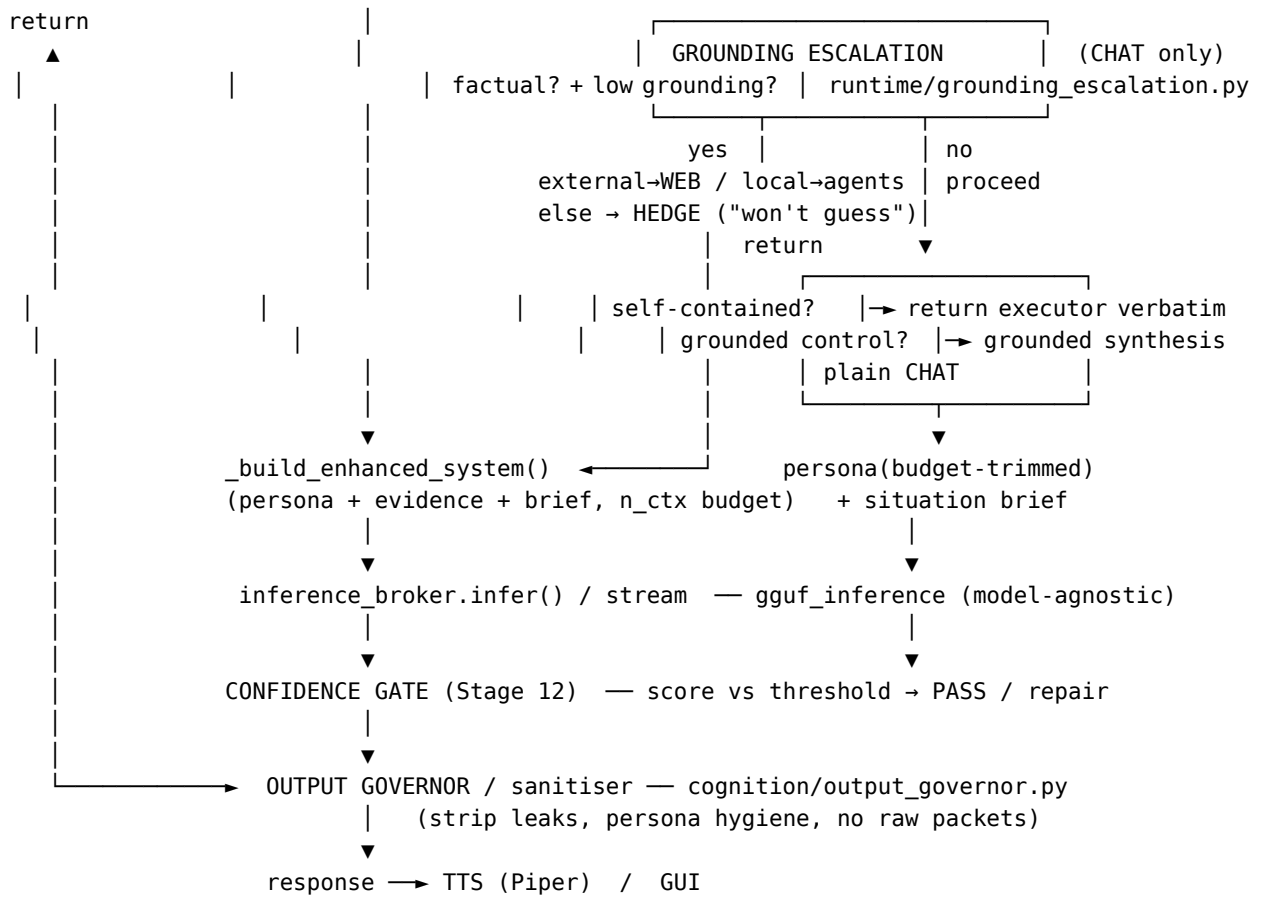
Blueprint — ELI MKXI ASCII Diagrams	1
1. Full pipeline (request lifecycle)	1
2. Memory subsystem eli/memory + cognition/working_memory.py	2
3. Gating stack (in path order — every guard the request passes)	3
4. One-screen overview (how it all connects)	4

Blueprint — ELI MKXI ASCII Diagrams

Visual companion to architecture.md. Three views: the full request pipeline, the memory subsystem, and the gating stack. All grounded in the real modules (file paths are clickable).

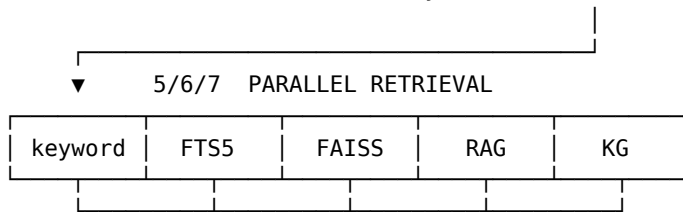
1. Full pipeline (request lifecycle)





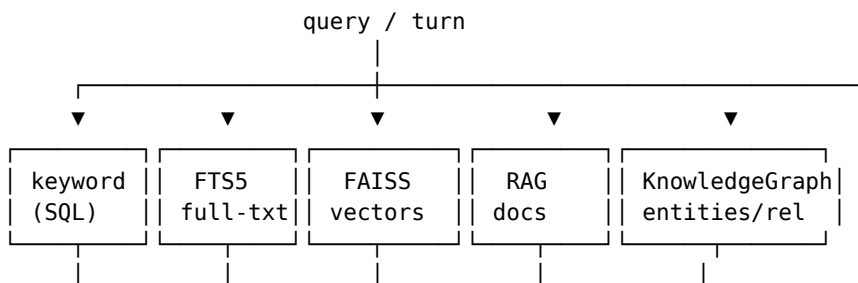
1b. Orchestrator — the 12 stages (path [B]) cognition/orchestrator.py

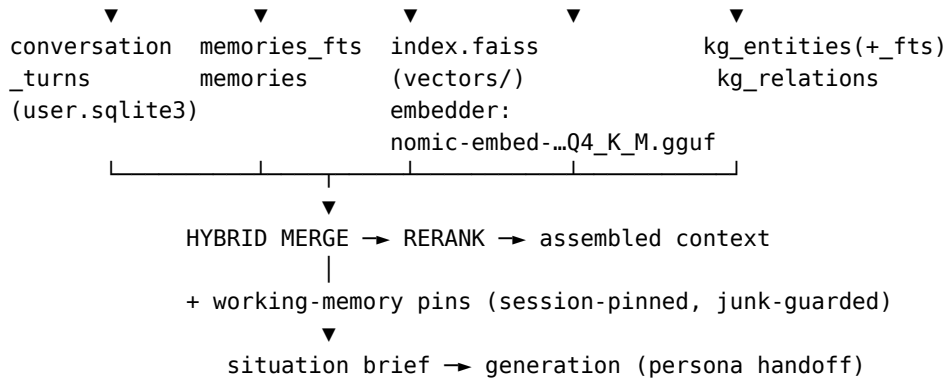
1 Intent → 2 Persona Lock → 3 HyDE → 4 Planner



8 Hybrid Merge → 9 Rerank → 10 Context Assembly
→ 10.5 Persona Handoff → 11 LLM Generation → 12 Confidence

2. Memory subsystem eli/memory + cognition/working_memory.py





STORES (artifacts/)	
db/user.sqlite3	memories(+fts), conversation_turns, conversations, kg_entities(+fts), kg_relations, recall_log, runtime_events, news_articles(+fts), news_reflections, habits/habit_events/habit_rules, observations, learning_replay, working_memory_pins, user_patterns, session_summaries, corrections, failures
db/agent.sqlite3	agent_dispatches, agent_metrics, improvements, failures, code_patches, error_tracking, observations
vectors/index.faiss	semantic index
runtime/users/<uuid>/user_profile...	per-user profile

WRITE PATH: turn → PERSISTENCE GATE (drop junk/report-dumps) → store

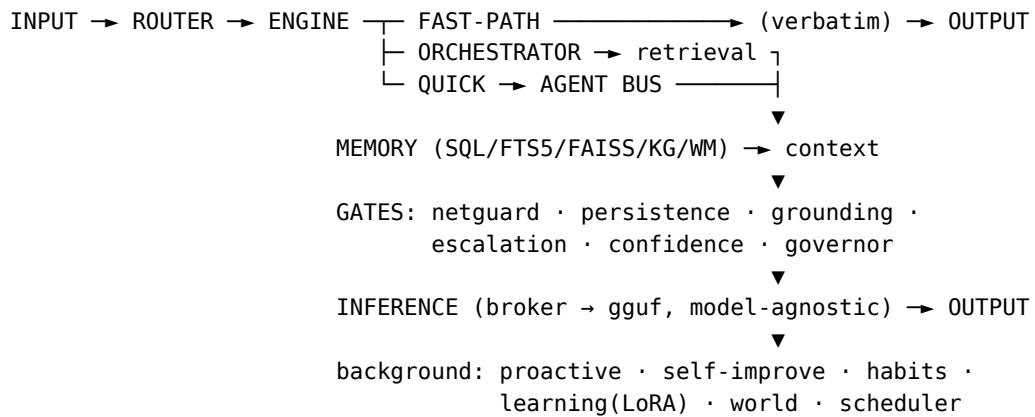
3. Gating stack (in path order — every guard the request passes)

1	NETGUARD (process-wide, always-on)	core/netguard.py	
	any outbound socket → offline? → OfflineError (FAIL-CLOSED)		
	allow_network(): scoped opt-in window (model download, web tier)		
2	PHASE45 FAST-PATH GATE	kernel/engine.py	
	deterministic action (volume/media/date/job/status)?		
	→ execute_action() VERBATIM, no LLM (cannot confabulate)		
3	PERSISTENCE GATE	runtime/persistence_gate.py	
	about to write to memory? junk / internal report-dump?		
	→ DROP (so it can't be stored and replayed as fact later)		
4	GROUNDING GATE	runtime/deterministic_gate	
	answer requires evidence? unbacked or raw packet?		
	→ BLOCK raw evidence/telemetry leak; require grounded synthesis		
5	GROUNDING ESCALATION	runtime/grounding_escalation	
	checkable fact AND grounding < threshold ?		
	external fact → WEB tier		
	self/project → LOCAL tier		
	exhausted / offline		
	first tier that grounds → answer		
	→ HEDGE ("I won't guess")		
	(trigger = GROUNDING, not the response score – which lies when wrong)		

6	CONFIDENCE GATE (Stage 12)	kernel/engine.py
	response score vs threshold → PASS / repair pass	
7	OUTPUT GOVERNOR / SANITISER	cognition/output_governor.py
	final text → strip leaks, persona hygiene, no raw JSON → user	

Legend: FAIL-CLOSED = denies by default; VERBATIM = returned without an LLM pass; HEDGE = honest "can't verify" instead of a guess.

4. One-screen overview (how it all connects)



“